

Idea for my bachelor's thesis: *VLM in Flash*

Seol Handong

Under the displayed upper-bound formulation, an exact optimum can always be represented by a single contiguous chunk.

Problem setup

Let N be the number of input channels of a weight matrix and let

$$M \in \{0, 1\}^N,$$

where $M_i = 1$ means that channel i and its corresponding weight row are selected.

Let $\mathcal{C}(M)$ denote the set of maximal contiguous runs of selected channels.

Define

$$R(M) = \|M\|_1$$

as the number of selected rows and

$$K(M) = |\mathcal{C}(M)|$$

as the number of maximal contiguous chunks. Under the paper's additive latency model,

$$\widehat{L}(M) = \sum_{C \in \mathcal{C}(M)} T[|C|],$$

where $T[r]$ is the profiled flash-read latency associated with a contiguous chunk of r rows.

Define the total retained activation importance as

$$I(M) = \sum_{i=0}^{N-1} v_i M_i,$$

where v_i is the importance of channel i . The paper's displayed selection problem can then be written as

$$\boxed{\max_{\substack{M \in \{0,1\}^N \\ 1 \leq R(M) \leq R}} \frac{I(M)}{\widehat{L}(M)}}, \quad R = \lfloor (1 - \rho)N \rfloor.$$

The lower bound excludes the empty mask, for which the ratio is undefined.

For a contiguous chunk C , define

$$I(C) = \sum_{i \in C} v_i, \quad U(C) = \frac{I(C)}{T[|C|]}.$$

Here $U(C)$ is the chunk's retained importance per unit of estimated flash-read latency.

Single-chunk optimality

Because distinct maximal chunks partition the selected channels,

$$I(M) = \sum_{C \in \mathcal{C}(M)} I(C).$$

Therefore,

$$\frac{I(M)}{\widehat{L}(M)} = \frac{\sum_{C \in \mathcal{C}(M)} I(C)}{\sum_{D \in \mathcal{C}(M)} T[|D|]} = \sum_{C \in \mathcal{C}(M)} \underbrace{\frac{T[|C|]}{\sum_{D \in \mathcal{C}(M)} T[|D|]}}_{w_C} \underbrace{\frac{I(C)}{T[|C|]}}_{U(C)}.$$

The coefficients satisfy

$$w_C \geq 0, \quad \sum_{C \in \mathcal{C}(M)} w_C = 1.$$

Hence the mask-level importance–latency ratio is a latency-weighted average of the utilities of its constituent chunks:

$$\boxed{\frac{I(M)}{\widehat{L}(M)} \leq \max_{C \in \mathcal{C}(M)} U(C)}.$$

Let C^* be the highest-utility contiguous interval whose length does not exceed R :

$$C^* \in \arg \max_{\substack{C \text{ contiguous} \\ 1 \leq |C| \leq R}} \frac{I(C)}{T[|C|]}.$$

The mask selecting only C^* is feasible and attains

$$\frac{I(C^*)}{T[|C^*|]} = U(C^*).$$

Consequently,

$$\max_{\substack{M \in \{0,1\}^N \\ 1 \leq R(M) \leq R}} \frac{I(M)}{\widehat{L}(M)} = \max_{\substack{C \text{ contiguous} \\ 1 \leq |C| \leq R}} U(C).$$

Thus, under the displayed objective and additive latency assumption, there always exists an exact optimum consisting of only one contiguous chunk. Multiple chunks can also be optimal in the exceptional case where every selected chunk has the same maximum utility.

Exact search with prefix sums

This optimum can be found without examining all 2^N masks. Define the importance prefix sums

$$P_0 = 0, \quad P_j = \sum_{i=0}^{j-1} v_i.$$

A chunk

$$C_{i,\ell} = \{i, i+1, \dots, i+\ell-1\}$$

then has

$$I(C_{i,\ell}) = P_{i+\ell} - P_i$$

and utility

$$U(i, \ell) = \frac{P_{i+\ell} - P_i}{T[\ell]}.$$

There are

$$\sum_{\ell=1}^R (N - \ell + 1) = \frac{R(2N - R + 1)}{2}$$

contiguous intervals of length at most R . Without the length restriction, the total number is

$$\binom{N+1}{2} = \frac{N(N+1)}{2}.$$

Prefix sums require $O(N)$ preprocessing and make each interval score computable in $O(1)$. The exact optimum is therefore obtainable in

$$\boxed{O(NR)}$$

time and $O(N)$ memory, which becomes $O(N^2)$ time in the worst case $R = \Theta(N)$.

Mismatch between the formulation and implementation

This simple exact solution exposes a mismatch between the displayed formulation and the implemented algorithm. The displayed constraint

$$R(M) \leq R$$

permits the optimizer to leave most of the budget unused, whereas the paper's greedy procedure continues accepting chunks toward R . Its implemented behavior is therefore closer to a fixed-cardinality or cardinality-band problem such as

$$R(M) = R$$

or

$$R_- \leq R(M) \leq R_+,$$

for which the maximum-utility single-chunk argument no longer provides a general exact solution.

The paper's greedy algorithm approximates a fixed-budget importance-per-latency objective.

Fixed-budget interpretation

The paper's implementation treats

$$R = \lfloor (1 - \rho)N \rfloor$$

as a target number of weight rows to retain. Rather than deciding the sparsity level, the selection algorithm receives R from the sparsity allocation and determines which rows should be loaded.

Let $\mathcal{G}$ denote the generated set of contiguous candidate chunks. Each candidate $C \in \mathcal{G}$ is assigned the utility

$$U(C) = \frac{I(C)}{T[|C|]}, \quad I(C) = \sum_{i \in C} v_i.$$

Thus, $U(C)$ measures the activation importance preserved by the chunk per unit of estimated flash-read latency.

Greedy selection rule

Let $G = |\mathcal{G}|$ be the number of generated candidates. They are sorted by decreasing utility:

$$U(C_{(1)}) \geq U(C_{(2)}) \geq \dots \geq U(C_{(G)}).$$

The algorithm initializes

$$S \leftarrow \emptyset, \quad q \leftarrow 0,$$

where S is the set of accepted chunks and q is the number of selected rows. It then processes the sorted candidates in order. A candidate $C_{(j)}$ is accepted if

$$C_{(j)} \cap \bigcup_{D \in S} D = \emptyset$$

and

$$q + |C_{(j)}| \leq R.$$

Upon acceptance,

$$S \leftarrow S \cup \{C_{(j)}\}, \quad q \leftarrow q + |C_{(j)}|.$$

The procedure continues until $q = R$ or no remaining candidate can be accepted. The resulting mask is

$$M_i^{\text{greedy}} = \mathbf{1} \left\{ i \in \bigcup_{C \in S} C \right\}.$$

The algorithm may return fewer than R rows when the remaining budget cannot be filled by any nonoverlapping candidate. Nevertheless, its operational goal is to select approximately R rows rather than to determine the row count from scratch.

Relationship to the global ratio

The candidate utility is the chunk-level counterpart of the mask-level objective

$$\frac{I(M)}{\widehat{L}(M)} = \frac{\sum_{C \in \mathcal{C}(M)} I(C)}{\sum_{C \in \mathcal{C}(M)} T[|C|]}.$$

Accordingly, the algorithm gives priority to chunks with high importance per estimated latency and greedily combines them while attempting to fill the prescribed row budget. This is a heuristic construction: ranking chunks individually does not guarantee that their combination maximizes the global ratio.

Comparison with top- R sparsification

The position of Neuron Chunking can be understood by comparing it with conventional top- R sparsification. Given the same prescribed row count, top- R selects

$$M_{\text{top-}R} \in \arg \max_{\substack{M \in \{0,1\}^N \\ R(M)=R}} I(M).$$

Because

$$I(M) = \sum_i v_i M_i,$$

this selects the R channels with the largest individual importance values. It maximizes retained activation importance without considering the physical arrangement or flash-read latency of the selected rows.

Neuron Chunking instead aims to balance retained importance against flash latency:

$$M_{\text{chunk}} \approx \arg \max_{\substack{M \in \{0,1\}^N \\ R(M)=R}} \frac{I(M)}{\widehat{L}(M)}.$$

The symbol $\approx$ indicates that the paper's greedy procedure is intended as a fast heuristic for this objective, not as an exact optimizer.

The methodological difference is therefore

$$\begin{array}{ll} \text{Top-}R : & \max_{R(M)=R} I(M), \\ \text{Neuron Chunking} : & \max_{R(M)=R} \frac{I(M)}{\widehat{L}(M)}. \end{array}$$

Top- R always prefers the individually most important rows, even when they are scattered across storage. Neuron Chunking may instead select contiguous, slightly less important rows when the reduction in flash-read latency compensates for the lost importance.

Displayed constraint versus reconstructed objective

Section 3.2.1 formally states the constraint as

$$R(M) \leq R.$$

The fixed- R expression above is therefore not the literal displayed formulation. It is a reconstruction of the problem suggested by the algorithm and experimental comparison: R is determined by the target sparsity, and the greedy procedure continues selecting chunks toward that prescribed budget.

The profiled chunk latency is closely approximated by an affine function.

Affine fit of the released latency profiles

The paper represents the latency of reading a contiguous chunk using the profiled lookup table

$$T[r],$$

where r is the number of weight rows in the chunk. The released profiles instead record chunk size in kilobytes. Let z denote the chunk size in KB. A least-squares fit to the released latency tables gives

$$T_{\text{KB}}(z) \approx a + c_{\text{KB}}z.$$

The fitted models are:

Device	Profiled range	Affine fit, with z in KB and T in ms	R^2
Jetson Orin AGX	1–255 KB	$T(z) \approx 0.01100 + 0.0001398z$	0.9867
Jetson Orin Nano	1–350 KB	$T(z) \approx 0.01573 + 0.0002842z$	0.9940

These R^2 values are not reported in the original paper. They are obtained by fitting the latency tables released with its implementation. The mean absolute percentage errors are approximately 4.22% on AGX and 3.21% on Nano.

The latency curves therefore have an approximately linear increasing shape with a positive intercept and relatively small local fluctuations. The affine approximation is intended as a structural model rather than an exact replacement for every entry in the lookup table.

Validation in the throughput domain

The model can also be validated in the throughput domain. The released implementation computes logical throughput as

$$\beta(z) = \frac{1000z}{1024T_{\text{KB}}(z)} \quad \text{MiB/s},$$

where z is measured in KB and $T_{\text{KB}}(z)$ in milliseconds. Substituting the affine latency model gives

$$\hat{\beta}(z) = \frac{1000z}{1024(a + c_{\text{KB}}z)}.$$

This function increases with chunk size and satisfies

$$\lim_{z \rightarrow \infty} \hat{\beta}(z) = \frac{1000}{1024c_{\text{KB}}} \quad \text{MiB/s}.$$

It therefore reproduces the qualitative behavior observed in the paper: throughput rises rapidly for small contiguous reads and gradually approaches a bandwidth ceiling.

The throughput curves derived from the affine model also closely match the released measurements. Their coefficients of determination are

$$R_{\text{AGX}}^2 = 0.9875, \quad R_{\text{Nano}}^2 = 0.9866.$$

At the saturation sizes reported in Appendices D and H,

$$z_{\text{sat}}^{\text{AGX}} = 236 \text{ KB}, \quad z_{\text{sat}}^{\text{Nano}} = 348 \text{ KB},$$

the measured and affine-predicted throughputs are approximately:

Device	Measured throughput	Affine prediction
AGX	5221 MiB/s	5238 MiB/s
Nano	3096 MiB/s	2965 MiB/s

The fitted asymptotic throughputs are

$$\beta_{\infty}^{\text{AGX}} \approx 6985 \text{ MiB/s}, \quad \beta_{\infty}^{\text{Nano}} \approx 3436 \text{ MiB/s}.$$

For comparison, the advertised SSD limits are

$$7450 \text{ MB/s} \approx 7105 \text{ MiB/s}$$

for AGX and

$$3500 \text{ MB/s} \approx 3338 \text{ MiB/s}$$

for Nano. The fitted asymptotes differ from these advertised limits by approximately 1.7% and 2.9%, respectively.

The asymptotic comparison should be interpreted cautiously. The quantities β_{∞} are extrapolated limits of the fitted models, whereas the paper measures throughput only at finite chunk sizes. Nevertheless, the agreement in both the latency and throughput domains supports the affine approximation over the profiled operating range.

Conversion from kilobytes to weight rows

To express the model in weight-row units, suppose one weight row occupies b KB. A chunk containing r rows then occupies

$$z = br \text{ KB}.$$

Therefore,

$$T[r] = T_{\text{KB}}(br) \approx a + c_{\text{KB}} br.$$

Defining

$$c = bc_{\text{KB}},$$

the row-based latency model becomes

$$T[r] \approx a + cr.$$

Here:

- a is the effective fixed latency incurred once per contiguous chunk;
- c is the marginal latency of adding one more weight row to that chunk.

The parameter a summarizes the effective penalty associated with small independent reads. It is inferred from the affine fit rather than separately measured by the paper as the cost of one particular hardware operation.

Consequences for mask-level latency

Substituting the affine approximation into the paper's additive mask-level latency model gives

$$\begin{aligned}\widehat{L}(M) &= \sum_{C \in \mathcal{C}(M)} T[|C|] \\ &\approx \sum_{C \in \mathcal{C}(M)} (a + c|C|) \\ &= \boxed{aK(M) + cR(M)}.\end{aligned}$$

Thus, under the affine approximation, mask latency depends only on

- the number of separate contiguous chunks, $K(M)$;
- the total number of selected rows, $R(M)$.

When the number of selected rows is fixed at R ,

$$R(M) = R,$$

the row-transfer term becomes constant:

$$\boxed{\widehat{L}(M) \approx aK(M) + cR.}$$

For $a > 0$, masks containing the same number of rows are therefore ordered by their numbers of chunks:

$$\boxed{R(M_1) = R(M_2) = R \implies \widehat{L}(M_1) < \widehat{L}(M_2) \iff K(M_1) < K(M_2).}$$

Equivalently,

$$\arg \min_{\substack{M \in \{0,1\}^N \\ R(M)=R}} \widehat{L}(M) = \arg \min_{\substack{M \in \{0,1\}^N \\ R(M)=R}} K(M).$$

This explains why the paper emphasizes long contiguous regions. Among masks containing the same number of rows, a mask with fewer and longer chunks pays the effective per-chunk cost a fewer times than a fragmented mask.

The fixed-budget affine objective admits an exact polynomial-time dynamic program.

Fixed-budget formulation and DP state

Substituting the affine latency model into the reconstructed fixed-budget objective gives

$$\max_{\substack{M \in \{0,1\}^N \\ R(M)=R}} \frac{I(M)}{aK(M) + cR}.$$

Although there are 2^N possible masks, exhaustive enumeration is unnecessary. The selected channels form a one-dimensional binary sequence, and the number of chunks can be written as

$$K(M) = M_1 + \sum_{i=2}^N (1 - M_{i-1})M_i.$$

A new chunk begins exactly when a selected channel follows either the beginning of the sequence or an unselected channel. Consequently, the effect of selecting channel i depends only on whether channel $i - 1$ was selected. This local dependency allows dynamic programming over the channel chain.

For notational convenience, index the channel importances by $v_1, \dots, v_N$. Define

$$F_i^s(r, k)$$

as the maximum importance obtainable from the first i channels when

- exactly r channels have been selected;
- exactly k chunks have been formed;
- channel i has selection state $s \in \{0, 1\}$.

The initial conditions are

$$F_0^0(0, 0) = 0,$$

with every other initial state set to $-\infty$.

Dynamic-programming recurrence

If channel i is not selected, the previous sequence may end in either state:

$$F_i^0(r, k) = \max \{F_{i-1}^0(r, k), F_{i-1}^1(r, k)\}.$$

If channel i is selected, it either continues an existing chunk or starts a new one:

$$F_i^1(r, k) = v_i + \max \{F_{i-1}^1(r-1, k), F_{i-1}^0(r-1, k-1)\}.$$

The maximum importance achievable with exactly R selected rows and k chunks is therefore

$$I^*(R, k) = \max_{s \in \{0,1\}} F_N^s(R, k).$$

The exact optimum of the affine fixed-budget problem is

$$\boxed{\max_{1 \leq k \leq K_{\max}} \frac{I^*(R, k)}{ak + cR}},$$

where

$$K_{\max} = \min(R, N - R + 1)$$

is the largest possible number of chunks in a binary mask containing exactly R selected positions.

Exact importance–latency frontier

The dynamic program yields more than the single ratio-maximizing mask. For every feasible chunk count k , it gives the largest importance achievable with exactly R selected rows. Since the affine latency associated with that state is

$$L_R(k) = ak + cR,$$

the points

$$\boxed{\mathcal{F}_R = \{(ak + cR, I^*(R, k)) : 1 \leq k \leq K_{\max}\}}$$

trace the exact fixed- R importance–latency trade-off under the affine model.

Increasing k strictly increases affine latency when $a > 0$. It also gives the optimizer greater freedom to collect important rows from separated regions, so the achievable importance generally increases along the upper envelope of these points. The exact- k values $I^*(R, k)$ need not be monotone for every activation vector because the feasible sets for different exact values of k are not nested. If a monotone envelope is desired, define

$$I_{\leq}^*(R, k) = \max_{1 \leq j \leq k} I^*(R, j).$$

The nondominated points in $\mathcal{F}_R$ form the fixed-budget Pareto frontier. The paper's greedy result,

$$\left(\widehat{L}(M_{\text{greedy}}), I(M_{\text{greedy}}) \right),$$

can be plotted against this frontier. This reveals how far the greedy mask lies from the best importance achievable at comparable affine latency. If the greedy procedure underfills the budget, it should instead be compared with the frontier corresponding to its realized row count $R(M_{\text{greedy}})$.

The dynamic program has time complexity

$$O(NRK_{\max})$$

and rolling-array memory complexity

$$O(RK_{\max}).$$

Since $R \leq N$ and $K_{\max} \leq N$, this is polynomial rather than exponential, with a worst-case time complexity of $O(N^3)$. The recurrence therefore provides an exact oracle in principle. For full-scale projections, however, its practical use requires either reduced instances or further acceleration.

Faster solution through fractional programming

A faster formulation eliminates the explicit chunk-count dimension through fractional programming. For a parameter $\eta \geq 0$, define

$$\Psi(\eta) = \max_{\substack{M \in \{0,1\}^N \\ R(M)=R}} [I(M) - \eta (aK(M) + cR)].$$

Let

$$\eta^* = \max_{\substack{M \in \{0,1\}^N \\ R(M)=R}} \frac{I(M)}{aK(M) + cR}.$$

Then

$$\Psi(\eta) \begin{cases} > 0, & \eta < \eta^*, \\ = 0, & \eta = \eta^*, \\ < 0, & \eta > \eta^*. \end{cases}$$

For a fixed η , define $G_i^s(r; \eta)$ as the maximum value of

$$I(M) - \eta a K(M)$$

over the first i channels, with r selected channels and final state s .

The chain-DP transitions are

$$G_i^0(r; \eta) = \max \{ G_{i-1}^0(r; \eta), G_{i-1}^1(r; \eta) \},$$

and

$$G_i^1(r; \eta) = v_i + \max \{ G_{i-1}^1(r-1; \eta), G_{i-1}^0(r-1; \eta) - \eta a \}.$$

Continuing a selected run adds no new chunk cost, whereas changing from state 0 to state 1 starts a new chunk and incurs the penalty ηa .

After processing all channels,

$$\Psi(\eta) = \max_{s \in \{0,1\}} G_N^s(R; \eta) - \eta c R.$$

Dinkelbach's method repeatedly solves this parametric chain problem and updates

$$\eta_{t+1} = \frac{I(M_t)}{aK(M_t) + cR},$$

where M_t maximizes the transformed objective at parameter η_t . The procedure terminates when

$$\Psi(\eta_t) = 0$$

up to the desired numerical tolerance.

Each parametric chain-DP evaluation takes

$$O(NR)$$

time. If Dinkelbach's method performs q iterations, the total running time is

$$O(qNR),$$

with $O(R)$ memory for the objective values, excluding the additional storage required to reconstruct the mask.

The three-dimensional dynamic program provides a direct polynomial-time exact oracle, while the parametric chain-DP removes the explicit chunk-count dimension and is potentially much faster in practice. Both methods are exact for the fitted affine objective

$$\frac{I(M)}{aK(M) + cR},$$

but not automatically for the paper's original lookup-table objective when $T[r]$ deviates from $a + cr$.

An importance-coverage constraint defines an alternative optimization problem.

Coverage formulation

The paper's fixed-budget formulation prescribes the number of selected rows,

$$R(M) = R,$$

and searches for a mask with a favorable importance–latency ratio. The alternative formulation reverses these roles: it prescribes how much activation importance must be retained and minimizes the latency required to retain it.

Define the total activation importance as

$$I_{\text{tot}} = \sum_{i=1}^N v_i,$$

and let $\alpha \in (0, 1]$ be the target importance-retention ratio. The required importance is

$$B_\alpha = \alpha I_{\text{tot}}.$$

The proposed coverage problem is

$$\boxed{M_\alpha^* \in \arg \min_{M \in \{0,1\}^N} \widehat{L}(M) \quad \text{subject to} \quad I(M) \geq \alpha I_{\text{tot}}.}$$

The constraint is written as an inequality because the importance values are real-valued, so a mask satisfying the exact equality

$$I(M) = \alpha I_{\text{tot}}$$

may not exist.

Under the affine latency approximation,

$$\widehat{L}(M) \approx aK(M) + cR(M),$$

the problem becomes

$$M_{\alpha}^{\star} \in \arg \min_{M \in \{0,1\}^N} \{aK(M) + cR(M)\} \quad \text{subject to} \quad I(M) \geq \alpha I_{\text{tot}}.$$

Unlike the fixed-budget problem, this formulation does not prescribe $R(M)$. Both the number of selected rows and the number of chunks are determined by the optimization. Concentrated importance may allow the target to be reached with relatively few rows, whereas a smoother importance distribution may require selecting more rows.

Exact offline solution

The chain dynamic program developed for the fixed-budget problem can also solve the affine coverage problem exactly. For every feasible pair (r, k) , define

$$I^{\star}(r, k) = \max_{\substack{M \in \{0,1\}^N \\ R(M)=r \\ K(M)=k}} I(M) = \max_{s \in \{0,1\}} F_N^s(r, k).$$

Every mask with $R(M) = r$ and $K(M) = k$ has the same affine latency,

$$aK(M) + cR(M) = ak + cr.$$

Consequently, some mask with row count r and chunk count k satisfies the coverage constraint if and only if

$$I^{\star}(r, k) \geq \alpha I_{\text{tot}}.$$

The optimal counts are therefore

$$(r_{\alpha}^{\star}, k_{\alpha}^{\star}) \in \arg \min_{\substack{0 \leq r \leq N \\ 0 \leq k \leq K_{\max}(r) \\ I^{\star}(r, k) \geq \alpha I_{\text{tot}}}} (ak + cr),$$

where

$$K_{\max}(r) = \min(r, N - r + 1)$$

for $r \geq 1$, with $K_{\max}(0) = 0$.

The corresponding optimal mask can be recovered by backtracking through the DP state attaining (r_α^*, k_α^*) . Equivalently, the exact minimum latency is

$$L^*(\alpha) = \min_{\substack{r, k \\ I^*(r, k) \geq \alpha I_{\text{tot}}}} (ak + cr).$$

Computing the complete table $I^*(r, k)$ requires

$$O(N^3)$$

time in the unrestricted worst case and $O(N^2)$ rolling-array memory. Once computed, the same table can answer any number of coverage thresholds without rerunning the chain DP.

This algorithm is therefore suitable as an exact offline oracle. It establishes the best achievable latency under the affine model, but its cubic running time is unlikely to satisfy the paper's approximately 2 ms per-matrix online-selection target.

A fast Lagrangian chain solver

A faster online approximation can be obtained by relaxing the coverage constraint with a multiplier $\lambda \geq 0$:

$$\min_M [aK(M) + cR(M) + \lambda (B_\alpha - I(M))].$$

Because λB_α is constant for a fixed multiplier, the mask can be found by solving

$$\min_M [aK(M) + cR(M) - \lambda I(M)].$$

The multiplier λ controls the value assigned to retained importance:

- a small λ emphasizes low latency;
- a large λ rewards retaining more importance.

For a fixed λ , define $D_i^s(\lambda)$ as the minimum relaxed cost for the first i channels when channel i has state $s \in \{0, 1\}$. The initial conditions are

$$D_0^0(\lambda) = 0, \quad D_0^1(\lambda) = +\infty.$$

If channel i is not selected,

$$D_i^0(\lambda) = \min \{D_{i-1}^0(\lambda), D_{i-1}^1(\lambda)\}.$$

If channel i is selected, it either continues the current chunk or starts a new one:

$$D_i^1(\lambda) = c - \lambda v_i + \min \{D_{i-1}^1(\lambda), D_{i-1}^0(\lambda) + a\}.$$

The term $c - \lambda v_i$ is the marginal cost of selecting channel i . A transition from 0 to 1 additionally pays the per-chunk cost a , while a transition from 1 to 1 extends an existing chunk without paying a again.

This recurrence has only two states per channel. It does not explicitly track either $R(M)$ or $K(M)$. Therefore, one multiplier evaluation requires

$$O(N)$$

time. Evaluating q multiplier values requires

$$O(qN)$$

time, with $O(1)$ memory when only the optimal value is needed and $O(N)$ memory when the mask must be recovered.

A grid search or bisection-style search over λ can generate masks ranging from low-latency, low-importance solutions to higher-importance solutions. Among the generated masks satisfying

$$I(M) \geq \alpha I_{\text{tot}},$$

the feasible mask with the smallest actual affine latency is returned.

The $O(qN)$ complexity makes this method a plausible candidate for approaching the paper's online runtime target. Unlike Neuron Chunking, it requires neither multiscale candidate generation nor global candidate sorting. Its practical latency must nevertheless be established experimentally rather than inferred from asymptotic complexity alone.

The Lagrangian solver is not guaranteed to recover the exact coverage optimum. Linear scalarization can recover only supported points of the discrete importance–latency frontier. The minimum-latency mask satisfying a particular coverage threshold may be an unsupported point and may therefore fail to minimize the relaxed objective for every value of λ .

The two algorithms consequently serve different purposes:

Method	Complexity	Role
Exact (r, k) chain DP	$O(N^3)$	Offline optimality oracle
Lagrangian chain DP	$O(qN)$	Fast online approximation

The exact DP provides the ground truth against which the faster solver can be evaluated.

Importance–latency frontiers

Sweeping the coverage threshold produces the exact affine importance–latency frontier

$$\left\{ \left(\frac{I(M_\alpha^*)}{I_{\text{tot}}}, L^*(\alpha) \right) : \alpha \in \mathcal{A} \right\}.$$

A representative set of thresholds is

$$\mathcal{A} = \{0.80, 0.85, 0.90, 0.95, 0.97, 0.99\}.$$

If $\alpha_1 \leq \alpha_2$, every mask feasible at α_2 is also feasible at α_1 . Hence,

$$\alpha_1 \leq \alpha_2 \implies L^*(\alpha_1) \leq L^*(\alpha_2).$$

The curve therefore measures the minimum additional latency required to preserve progressively more importance. It does not imply that retaining more importance reduces latency.

The experiment can compare four sets of operating points on the same activation vectors:

- the exact coverage frontier obtained from the $O(N^3)$ DP;
- the $O(qN)$ Lagrangian solutions;
- the masks produced by Neuron Chunking;
- the masks produced by conventional top- R selection.

For any produced mask M , define its achieved coverage as

$$\alpha(M) = \frac{I(M)}{I_{\text{tot}}}.$$

Its relative gap from the exact coverage frontier is then

$$\text{Gap}(M) = \frac{\widehat{L}(M) - L^*(\alpha(M))}{L^*(\alpha(M))}.$$

Using the achieved coverage $\alpha(M)$, rather than the requested threshold, ensures that each method is compared with the exact optimum preserving at least as much importance as that method actually preserves.

Recording

$$R(M) \quad \text{and} \quad K(M)$$

at each operating point additionally reveals whether a latency reduction comes from selecting fewer rows, forming fewer chunks, or both.

The fixed- R and coverage frontiers answer distinct questions:

Frontier	Constraint	Variables allowed to change	Primary purpose
Fixed- R frontier	$R(M) = R$	$K(M)$ and selected positions	Evaluate the problem suggested by the paper’s implementation
Coverage frontier	$I(M) \geq \alpha I_{\text{tot}}$	$R(M)$, $K(M)$, and selected positions	Evaluate the alternative problem proposed here

Finally, activation importance remains a surrogate for model quality. Exact optimality on the importance–latency frontier does not imply exact optimality on the downstream accuracy–latency frontier. Selected operating points must therefore also be evaluated on the original VLM tasks.